Warsaw University of Technology
Faculty of Electronics and Information Technology

Institute of Computer Science

Postgraduate Studies
Big Data - Processing and Analysis of Large Data Sets

FINAL THESIS

Aleksandra Maria Biernat

Sentiment Analysis and Ranking of Reviews using Big Data Tools: Hadoop, Hive, Spark and Deep Learning

Thesis supervisor
Patryk Pilarski, MSc Eng.

Warsaw, 2021

ABSTRACT

The main goal of this project was a task of sentiment analysis in the context of Big Data, where based on a text review of a certain Kindle product, an overall mark given by a consumer is being predicted. For the purpose of this analysis, several models have been built and compared in order to ultimately choose two final ones. In this project, we explored Multinomial Logistic Regression and Gradient Boosted Tree – both done in Spark Scala in the Apache Hadoop ecosystem – as well as Deep Learning models which were trained making use of the keras and tensorflow libraries and GPU acceleration functionality available on Google Colab. Another explanatory feature investigated in the models was the 'category' feature with a category/genre that a certain Kindle book belongs to.

Moreover, a ranking system has been built also using the Apache Hadoop ecosystem and its component – Apache Hive. The latter is particularly useful in the e-commerce world, where potential clients would want to learn what the best products are, but not only taking into account either the mean review or popularity measure of a certain book, which have proved to be insufficient and often lead to only incomplete conclusions, but instead use a mixed measure of the two. This provides a better reflection as well as a fuller picture of an overall attractiveness and review of a certain book.

For all of the aforementioned tasks, relevant evaluation metrics have been obtained and compared or snapshots of results depicting the usefulness of a proposed approach were presented. Based on those, the best models and methods have been chosen and proposed as the final product. The possible applications and benefits of the presented models and methods have also been discussed and stated.

Keywords: Machine Learning, Apache Hadoop, Apache Hive, Ambari, Apache Spark, Deep Learning, Natural Language Processing, Ranking Systems, Recommendation Systems, E-commerce

TABLE OF CONTENTS

ABSTRACT 2
INTRODUCTION 4
- Chapter I: Pipeline 4
- Chapter II: Technology Choices 5

MAIN BODY 6
- Chapter I: ETL Process 6
  - Subchapter 1: Extracting Raw Data 6
  - Subchapter 2: Data Pre-processing (TRANSFORM) 7
  - Subchapter 3: Loading Data (LOAD) 8
- Chapter II: Data Exploration 10
  - Subchapter 1: Kindle Data 10
  - Subchapter 2: Meta Kindle Data 14
  - Subchapter 3: Merged Data After Initial Cleaning 15
  - Subchapter 4: Spark – Scala – Zeppelin (Hadoop) 17
- Chapter III: Data Cleaning 17
- Chapter IV: Modeling 20
  - Subchapter 1: Hive 20
  - Subchapter 2: Spark – Scala – Zeppelin (Hadoop) 22
  - Subchapter 3: Deep Neural Networks – Python – Google Colab 27
- Chapter V: Data-Driven Product 33
- Chapter VI: Final Conclusions 39

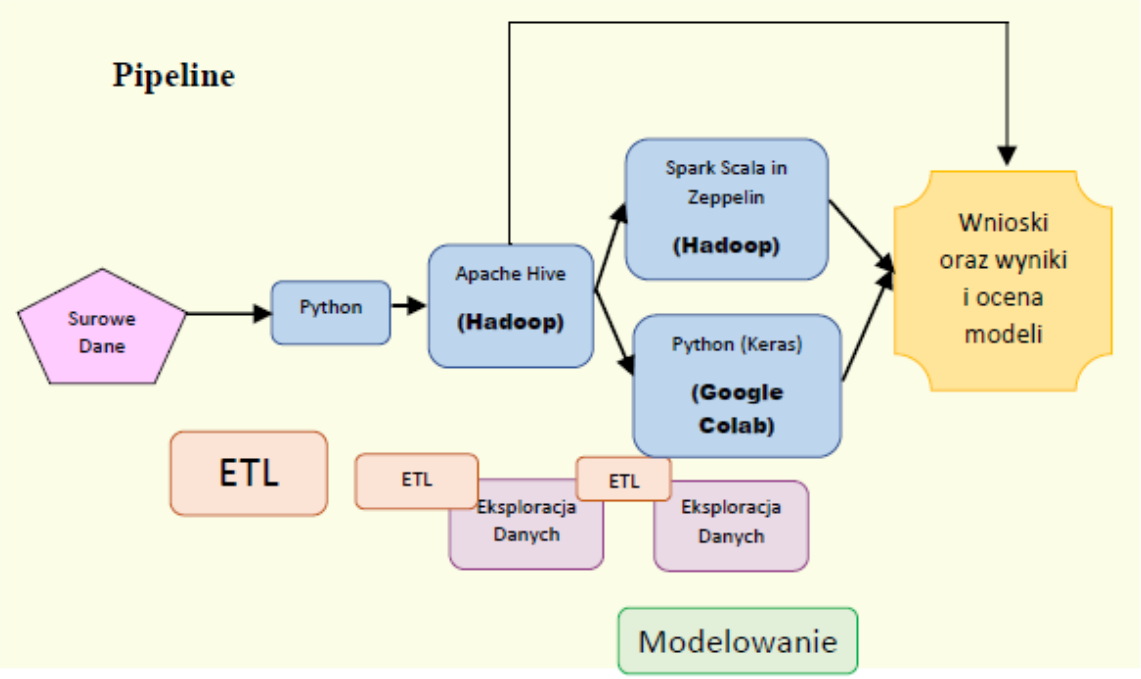
POTENTIAL NEXT STEPS 39
BIBLIOGRAPHY 40
APPENDICES 41

INTRODUCTION

Chapter I: Pipeline

An illustrative pipeline of the project is presented in Figure 1. It is a graphical representation of the main components of the process in this project, with smaller tasks such as data cleaning for a specific model performed at the corresponding stages. It should be noted that exploration, cleaning, and modeling processes often do not take place in a one-time manner after which we move to the next step. These are often iterative processes depending on conclusions drawn and dependencies that become apparent at various stages of building the process.

Figure 1: Illustrative project pipeline



Chapter II: Technology Choices

Python – Google Colab

Python was chosen because it is currently one of the most universal, widespread, and flexible programming languages. It also has wide application in machine learning, thanks to numerous packages and libraries containing partially ready-made models. In the context of this project, thanks to its integration with packages such as keras and tensorflow, it seems to be the best choice for building deep neural network models. Python is also one of the languages that can be used on Google Colab – an open environment accessible through a web browser, created and managed by Google. This makes Google Colab a very convenient tool, especially since computational power – particularly the ability to compute using GPU and TPU – is often greater than that of home computers (including mine). Due to limited RAM capacity, the deep neural network models presented in this project could not be trained on my local home computer.

Apache Hadoop

The Apache Hadoop ecosystem is one of the most commonly used tools for processing large data sets. It is useful because it is integrated with many different technologies, such as Hive (Tez), Spark SQL for data processing; Kafka, Spark Streaming for stream data processing; Spark, Zeppelin – useful in machine learning processes, automation, and data processing, among others. Through HDFS it also offers a file storage system, as well as database solutions such as HBase. A significant part of this project – through the use of Hive (an interface to Hadoop), Spark, and Zeppelin – was carried out in the Apache Hadoop ecosystem.

Ambari – Hortonworks Data Platform

An Apache Hadoop distribution created by Hortonworks and maintained by Cloudera. It is an open-source environment, which was the main reason for choosing this particular Apache Hadoop distribution. The installation process and version used are the same as explained in the Udemy MOOC course "The Ultimate Hands-On Hadoop: Tame your Big Data!" (Kane, n.d.).

Apache Hive

Regarded as an interface to Apache Hadoop. Its syntax resembles SQL. Like SQL environments, it is very convenient for regular data processing such as creating queries, aggregations, filtering, and data cleaning, which was the main motivation for using it in the project.

Apache Spark – Scala – Zeppelin

Apache Spark is a very fast engine created for large-scale data processing using distributed processing across different clusters. It is currently very popular, and access to knowledge about Spark is quite easy. Spark is integrated with other languages such as Java, Scala, and Python (so-called PySpark), and like Python, it is very flexible – offering a wide range of possibilities including machine learning (MLlib based on RDD, Spark ML based on DataFrame), graph analysis (GraphX), stream processing (Spark Streaming), and more "standard" data processing similar to SQL queries (Spark SQL). It is also faster than MapReduce, the original concept on which Hadoop was based. Scala is, however, the most native environment for Spark, and therefore the machine learning part in Spark was done in Spark Scala. Operations written in Scala are generally faster than in PySpark and consume fewer resources. Zeppelin is an environment where Spark Scala code can be written, enabling saving results and visualizations.

MAIN BODY

Chapter I: ETL Process

Subchapter 1: Extracting Raw Data (EXTRACT)

In this project, the data acquisition process is not very complex. The data comes from a single source (website) and all data is contained in only two files. The analyzed data represents reviews of e-books and products available on Kindle from the Amazon store (Jianmo Ni, 2019). The data is in JSON format and contained in two files: Kindle_Store.json (Ni, 2018) and meta_Kindle_Store.json (Ni, 2018) with sizes of 3.52 GB and 462 MB respectively.

According to the definition, for data to be considered Big Data (Szmit, n.d.), it must meet the "4V" condition: Volume (large amounts of data), Velocity (data grows rapidly), Variety (great diversity of data), and Value (high value of data). While by current Big Data standards, where data used by various companies often measures in tera- or petabytes, the data files analyzed in this project are not enormous; however, to create an effective tool for processing such data, Big Data techniques are at least very useful.

The data is in JSON format, specifically JSON Lines (also called newline-delimited JSON), where individual rows are in nested dictionary format. Keys correspond to table fields, and values to their values. This format is very convenient for structured and semi-structured data (as in this case), which can be processed line by line (JSON Lines Org., n.d.).

The data contains, among others: rating, ASIN (identification number assigned and used by Amazon), review text/opinion about the product, number of reviews, customer ID, review ID, and other less important fields.

Subchapter 2: Data Pre-processing (TRANSFORM)

The data transformation process mainly involved bringing data to a form that could be loaded into HDFS in the Hadoop Hortonworks Data Platform – Ambari distribution, and then loaded into Hive. The main goals were:

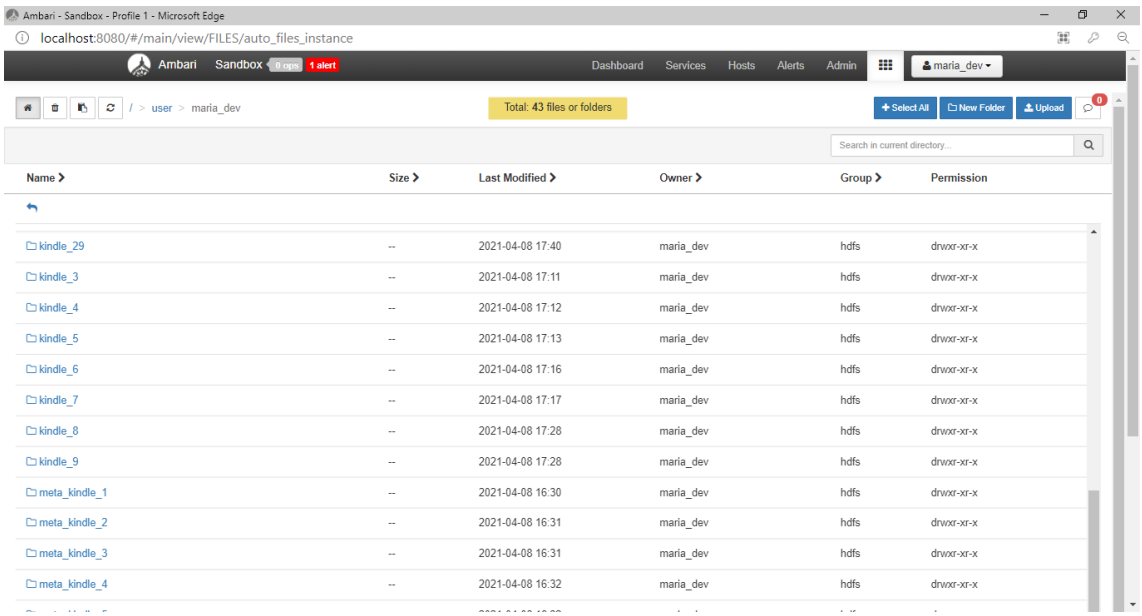
1. **Splitting files into smaller parts** (following the concept of sharding) – This was necessary due to the file size limit that can be uploaded to HDFS in Ambari at once.
2. **Initial data cleaning** – ensuring that data and special characters within it were compatible with Hive requirements. It was necessary to remove certain colons, as these are special characters used for identifying key-value cells when defining a table in Hive.

These steps were accomplished using code written in the object-oriented paradigm in Python in a Jupyter environment. As a result, Kindle_Store.json was split into 29 files, and meta_Kindle_Store.json into 5 files.

Subchapter 3: Loading Data (LOAD)

The first step necessary to analyze data in the Apache Hadoop ecosystem is loading data into HDFS (Hadoop Distributed File System). HDFS is a distributed file system that is part of the Apache Hadoop architecture (Szmit, n.d.) and serves as file storage. Files were uploaded so that each of the generated files was in a separate folder, as shown in Figure 2.

Figure 2: HDFS view in Ambari via HTTP showing individual folders with Kindle and Meta Kindle data



Next, tables were created in Apache Hive based on the uploaded files. Apache Hive is software that enables reading, writing, and general management, using SQL-like syntax, of large data sets stored in distributed file systems (e.g., HDFS) and databases integrated with Hadoop. Loading data into Apache Hive may require uploading an additional JSON SerDe library if the data in source files has a non-standard structure. The basic JSON SerDe library is usually already available. Thanks to this library, it is possible to deserialize data in JSON format.

Figure 3: DDL code using JSON SerDe to create a table based on data stored in HDFS

```
Create external table projectdb.meta_kindle_1(
  category ARRAY <string>,
  tech1 string,
  description ARRAY <string>,
  fit string,
  title string,
  also_buy ARRAY <string>,
  ...
  asin string
)
  row format serde 'org.apache.hive.hcatalog.data.JsonSerDe'
  LOCATION '/user/maria_dev/meta_kindle_1/';
```

In the most general distinction, Hive has two categories of data types: primitive and complex. Complex categories are divided into four subcategories: **Array** (an ordered sequence of similar-type elements), **Map** (a collection of key-value pairs), **Struct** (similar to C struct, can contain a set of fields of any primitive type), and **Union** (similar to union in C, not yet fully implemented in Hive).

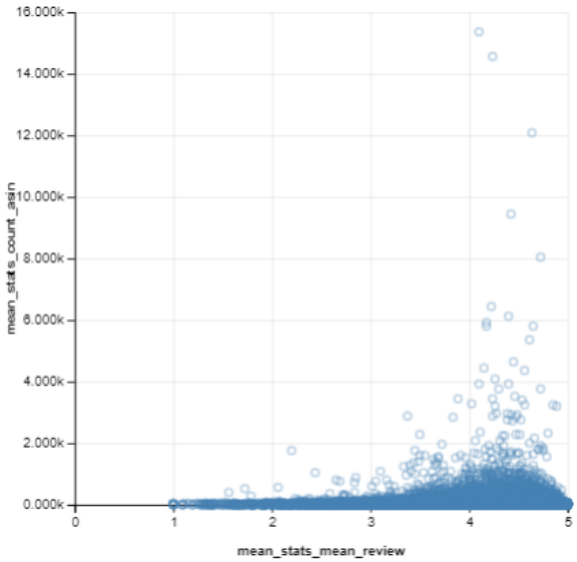
Chapter II: Data Exploration

Subchapter 1: Kindle Data

Initial exploratory data analysis was performed on a sample of the first 1,000 rows in the dataset – this is only an auxiliary analysis to determine what individual columns contain and to decide which data we want to keep for further analysis and which can already be removed. In the context of Big Data, such filtering should be done as early as possible to reduce data dimensionality, especially before performing computationally intensive operations such as joins.

Key columns identified: **overall** (rating 1–5), **verified** (purchase verification), **reviewerid** (reviewer ID), **asin** (Amazon product ID), **reviewtext** (review text), and **summary** (short review summary). Columns deemed unnecessary for analysis (style, reviewername, unixreviewtime) were marked for removal.

Figure 4: Relationship between mean number of reviews per e-book and its mean rating



Creating a ranking based solely on ratings is unreliable. Imagine two books with the same average rating: one has 10 reviews and the other 5,000. The second book is more popular, and should rank higher. We therefore need to incorporate review count information into the final measure used for hierarchizing products.

One possibility is to calculate a weighted average of ratings, where the weight is the number of reviews. However, this approach has complications. Figure 5 illustrates the distribution of review counts for different books in the dataset. This distribution resembles a lognormal distribution, where most observations are clustered around the left end, and few take disproportionately large values.

Figure 5: Distribution of review counts

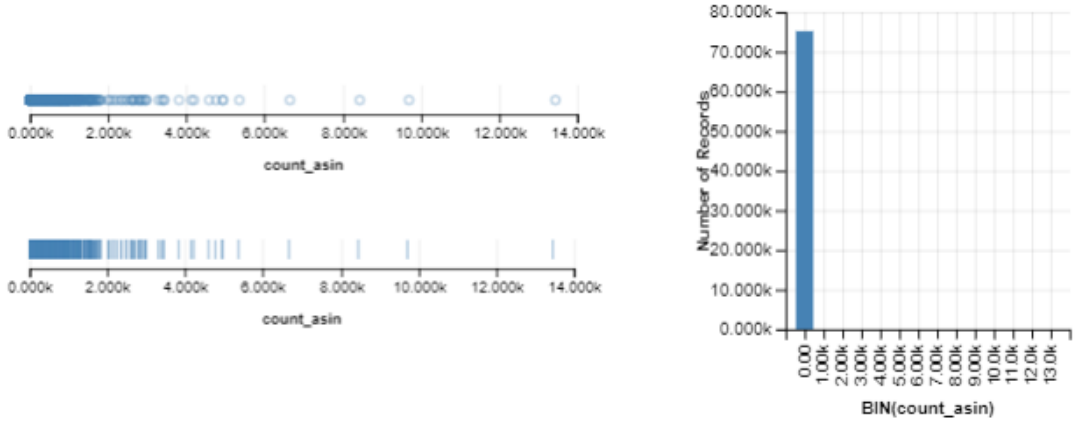
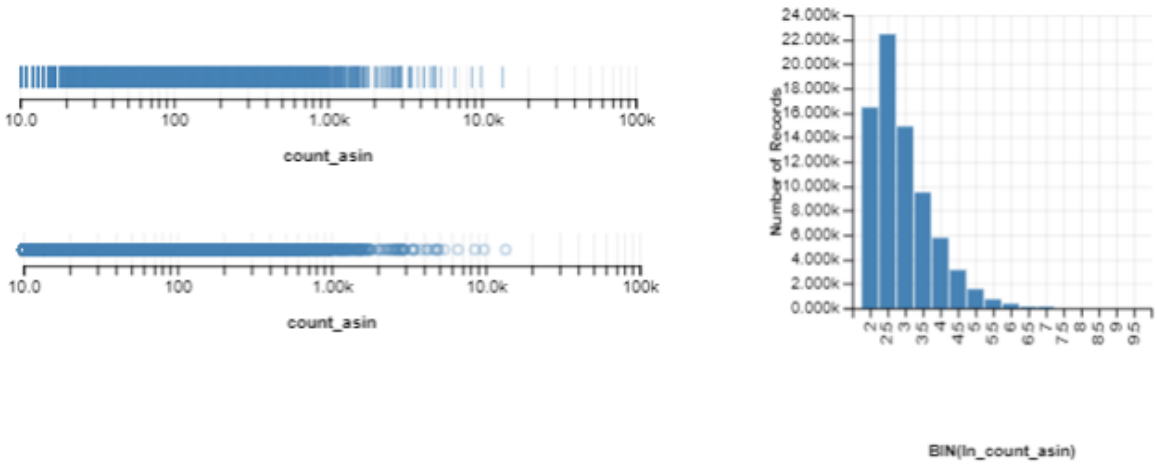


Figure 6: Distribution of review counts after log transformation



Using values from such a distribution as weights would cause final results to be very strongly correlated with it. An appropriate solution is to slightly change the shape of this distribution by using a natural logarithm transformation, which in theory converts a lognormal distribution to a normal distribution. The effect is illustrated by Figure 6 above.

Figure 7: Distribution of mean rating per book

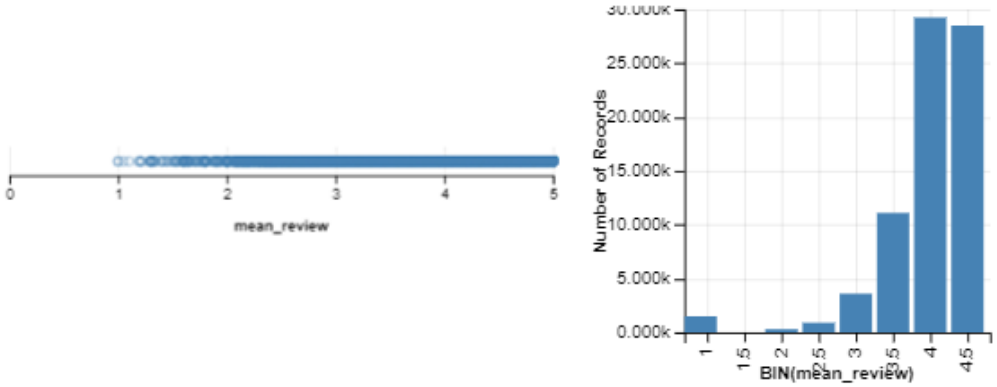
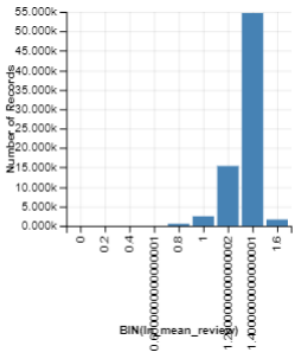


Figure 8: Distribution of mean rating after log transformation



Subchapter 2: Meta Kindle Data

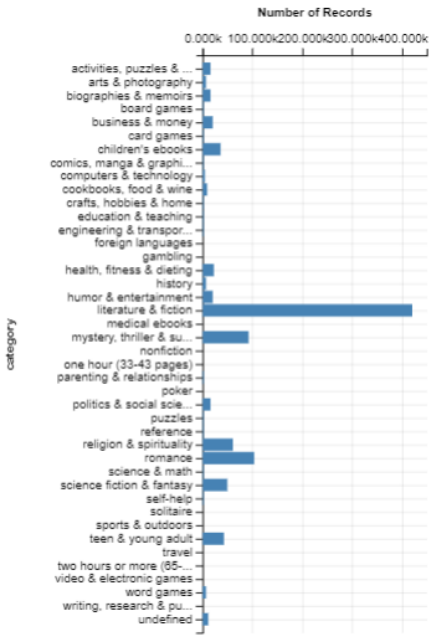
Similarly to the Kindle data, initial EDA was performed on a sample of the first 1,000 rows. Key column identified: **category** – containing all categories to which a given product belongs in string list format (e.g., ["Kindle Store","Kindle eBooks","Arts & Photography"]). The category a book belongs to is useful and will be one of the variables in the prediction model. Other columns (tech1, description, fit, also_buy, image, brand, rank, also_view, details, main_cat, similar_item, date, price) were analyzed but deemed unnecessary for this particular analysis.

Notably, the **also_buy** and **also_view** columns contain very interesting data that could be used for dependency analysis between e-books using, for example, a graph database like Neo4j or Spark's GraphX package.

Subchapter 3: Merged Data After Initial Cleaning

After the cleaning described in Chapter III, a test was performed to check for ASIN duplicates in the reduced_asin_hive_analysis table. The non-uniqueness of rows was caused by some ASIN numbers belonging to several categories. Most such cases concerned games rather than books. The number of unique ASIN numbers with duplicates was only 188. All such duplicate cases were therefore removed from the dataset.

Figure 9: Categories in the "raw" dataset – before removing duplicates

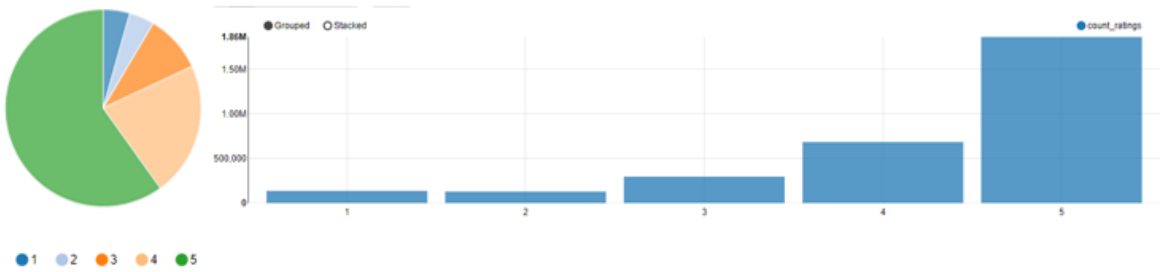


Subchapter 4: Spark – Scala – Zeppelin (Hadoop)

The data table used in this section is for_nlp_models.csv. In this section, we attempt to predict the rating of a title based primarily on the reviewtext variable, so this is a type of NLP (Natural Language Processing) analysis – specifically, sentiment analysis. An additional variable used for prediction is 'category'. Based on Figure 10, we can observe that the dataset is quite imbalanced. A significant

majority consists of ratings '5', followed by about 25% of ratings '4', and '3', '2', '1' making up the rest.

Figure 10: Distribution of ratings in the for_nlp_models.csv dataset



Chapter III: Data Cleaning

Subchapter 1: Kindle Data

Based on the exploratory analysis, the following cleaning steps were taken:

1. Only data with an ASIN number present was kept, since without the ASIN we cannot determine which book it is or join the corresponding category from the metadata.
2. Only books rated at least 10 times were selected for further analysis. Books with fewer reviews may be too niche or unpopular for reliable conclusions.
3. Only verified purchase reviews were kept, to avoid analyzing potentially unreliable reviews (e.g., those added by mistake or by competitors).
4. Calculation of mean review count and mean rating per book (per ASIN).
5. Cleaning the reviewtext column – if reviewtext is empty but text exists in the 'vote' column, the text was moved to reviewtext.

Subchapter 2: Meta Kindle Data

The following cleaning steps were applied:

1. Only fields ASIN, title, and category were retained.
2. The 'category' column required significant cleaning: it was exploded so individual list elements were placed in separate rows; converted to lowercase; filtered to retain only proper categories by removing certain keywords; and special characters were removed.

Figure 11: Data snippet – ASIN and category before processing

asin	category
B000FA5KKA	["Kindle Store","Kindle eBooks","Science Fiction & Fantasy"]
B000FA5M3K	["Kindle Store","Kindle eBooks","Engineering & Transportation",""]
B000FA5KJQ	["Kindle Store","Kindle eBooks","Biographies & Memoirs"]
B000FA5NSO	["Kindle Store","Kindle eBooks","Science Fiction & Fantasy"]
B000FA5KX2	["Kindle Store","Kindle eBooks","Business & Money"]

Figure 12: Data snippet – ASIN, title, and category after processing

meta_kindle_all_cat_v2.asin	meta_kindle_all_cat_v2.title	meta_kindle_all_cat_v2.category
B000FA5KKA	""	science fiction & fantasy
B000FA5M3K	""	engineering & transportation
B000FA5KJQ	""	biographies & memoirs
B000FA5NSO	""	science fiction & fantasy
B000FA5KX2	""	business & money
B000FA5M3U	""	business & money
B000FA5RE4	""	reference
B000FA5WK8	""	humor & entertainment
B000FA5VJK	""	history
B000FA5L2C	""	business & money
B000FA5O5Q	""	literature & fiction
B000FA5UXC	""	science fiction & fantasy
B000FA61JY	""	history

Subchapter 3: Final Steps

Both datasets (Kindle and meta_Kindle) were merged and two tables were created: **for_nlp_models** (columns: asin, overall, reviewtext, reviewerid, category) for building the sentiment

prediction model, and **reduced_asin_hive_analysis** (columns: asin, category, title, count_asin, mean_review) for the ranking analysis in Hive. Both datasets were saved in CSV format on HDFS.

Chapter IV: Modeling

Subchapter 1: Hive

A frequently interesting question from the consumer's perspective, and therefore from a business standpoint, is: *What products are rated highest?* On most websites, results can be sorted by popularity (most reviews) or highest rating (often 1 to 5). However, the product rating alone, without considering popularity, gives only a partial picture.

Consider two products: Product A with mean rating 5.0 and 1 review, vs. Product B with mean rating 4.5 and 800 reviews. It is hard to determine which is better using only one of these measures. The proposed solution is to create a mixed measure (such as a weighted average) that incorporates both rating and number of reviews.

As discussed earlier, using review count directly as a weight leads to complications due to the irregular, lognormal-like distribution. The approach taken was:

1. Apply natural log transformation to the review count to normalize the distribution.
2. Apply Min-Max scaling to map values to [0, 1].
3. Apply further scaling to map to [0.7, 1], reducing the impact of popularity relative to rating.

Equation 1: Min-Max Scaling

```
u = (x - min(x)) / (max(x) - min(x))
```

Equation 2: Scaling to [0.7, 1]

```
s = 0.7 + 0.3 * u
```

Equation 3: Final weighted average rating formula

```
rating(m, n) = m * (0.7 + 0.3 * u) = m * s

where m = mean_review, n = count_asin, x = ln(n)
```

Another method commonly used in e-commerce is constructing a mixed measure using Bayesian estimation. This approach takes into account the fact that we do not have enough data to base the estimate on the mean alone, and also incorporates the rest of the information we have about other data in the dataset. While a more systematic and stable approach, it was not further explored in this project, as the main goal is sentiment analysis. It is, however, a topic that could extend this work in the future.

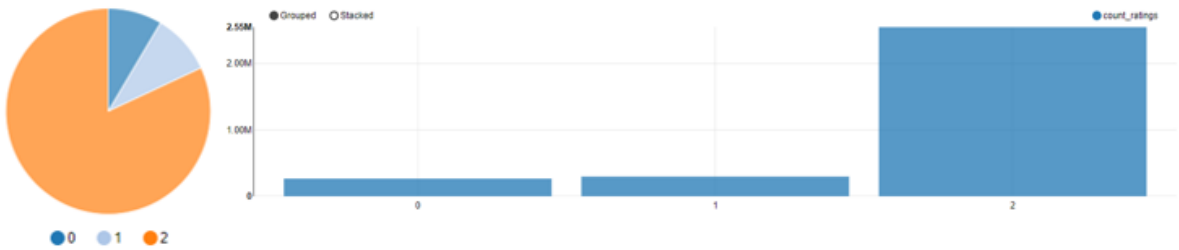
Subchapter 2: Spark – Scala – Zeppelin (Hadoop)

Predicting a rating based on review text and category requires appropriate preparation of explanatory variables. In this section, two models are compared: **Multinomial Logistic Regression (MLR)** – for multi-class target variables, and **Gradient Boosted Tree (GBT)** – which in the Spark version used requires a binary target variable.

MLR approach:

1. Target variable mapping: ratings 1,2 → 0; rating 3 → 1; ratings 4,5 → 2
2. Tokenization of reviewtext into individual words
3. Removal of stopwords
4. Word vectorization using TF-IDF with HashingTF
5. Converting categorical 'category' to numeric using StringIndexer
6. Combining variables using VectorAssembler()
7. Train/test split, model training with 3-fold cross-validation and hyperparameter grid search

Figure 13: Distribution of 'overall' variable after conversion for MLR model



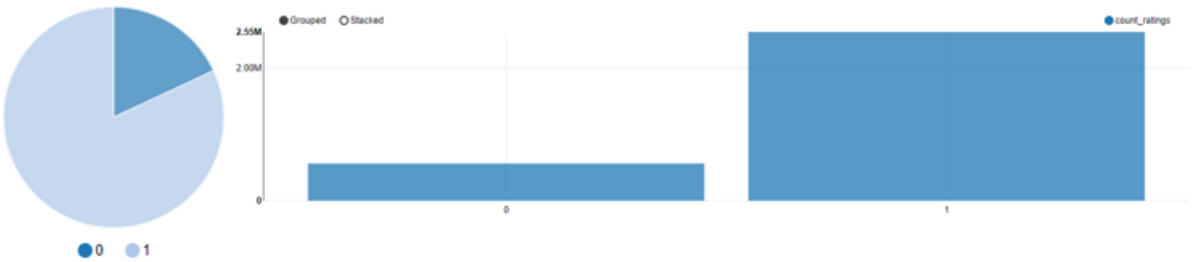
GBT approach:

1. Target variable mapping: ratings 1,2,3 → 0; ratings 4,5 → 1 (binary)

2-6. Same preprocessing steps as MLR

7. Model built with mostly default hyperparameters, without cross-validation

Figure 14: Distribution of 'overall' variable after conversion for GBT model



Model Selection:

Comparing the effect of data scale: MLR with all data (~3M) achieved F1 of 80.7%, while MLR with ~500K data achieved 80.1%. The difference is not significant, suggesting we are near the diminishing returns threshold.

Comparing variables used: MLR with only 'reviewtext' achieved F1 of 80.4%, while with both 'reviewtext' and 'category' it achieved 80.7%. The influence of category was negligible.

Figure 15: MLR results using only 'reviewtext' (~3M data)

```
Confusion matrix:
14995.0  2721.0  34848.0
3775.0   6757.0  48122.0
2900.0   3391.0  498820.0
accuracy: Double = 0.8446333046149054
Summary Statistics
Accuracy = 0.8446333046149054
Weighted precision: 0.8116540554906322
Weighted recall: 0.8446333046149055
Weighted F1 score: 0.8046790477510173
Weighted false positive rate: 0.6134454688876396
```

Figure 16: MLR results using 'reviewtext' and 'category' (~3M data)

```
Confusion matrix:
14125.0  2593.0  33628.0
3363.0   6536.0  46984.0
2692.0   3281.0  491108.0
accuracy: Double = 0.846865019609141
Summary Statistics
Accuracy = 0.846865019609141
Weighted precision: 0.8144684381243038
Weighted recall: 0.8468650196091411
Weighted F1 score: 0.8070515918639329
Weighted false positive rate: 0.6202997707641787
```

Figure 17: MLR results using 'reviewtext' and 'category' (~500K data)

```
// Weighted stats
println(s"Weighted precision: ${metrics.weightedPrecision}")
println(s"Weighted recall: ${metrics.weightedRecall}")
println(s"Weighted F1 score: ${metrics.weightedFMeasure}")
println(s"Weighted false positive rate: ${metrics.weightedFalsePositiveRate}")

import org.apache.spark.mllib.evaluation.MulticlassMetrics
results: org.apache.spark.sql.DataFrame = [asin: string, reviewtext: string ... 4 more fields]
predictionAndLabels: org.apache.spark.rdd.RDD[(Double, Double)] = MapPartitionsRDD[11096] at rdd at <console>:67
metrics: org.apache.spark.mllib.evaluation.MulticlassMetrics = org.apache.spark.mllib.evaluation.MulticlassMetrics@4bf6c4ff
Confusion matrix:
2372.0  461.0   5674.0
619.0   1071.0  7886.0
463.0   603.0   80501.0
accuracy: Double = 0.8423883592574009
Summary Statistics
Accuracy = 0.8423883592574009
Weighted precision: 0.8073653061777808
Weighted recall: 0.842388359257401
Weighted F1 score: 0.801804353087814
Weighted false positive rate: 0.6159478974618667
```

The GBT model achieved an auROC of only 54% – a very weak result. As the MLR model proved adequate and is computationally simpler, only MLR was considered in further analysis.

Subchapter 3: Deep Neural Networks – Python – Google Colab

Motivation for choosing deep neural networks:

Deep neural networks are scalable: the more data used, the better results can be achieved, in contrast to traditional ML algorithms which typically reach a plateau. They excel at detecting complex structures and patterns in data, as well as sequences, making them ideal for NLP and sentiment analysis tasks.

Figure 18: Jeff Dean's talk in Seoul (Dean, 2016) – Neural network scaling

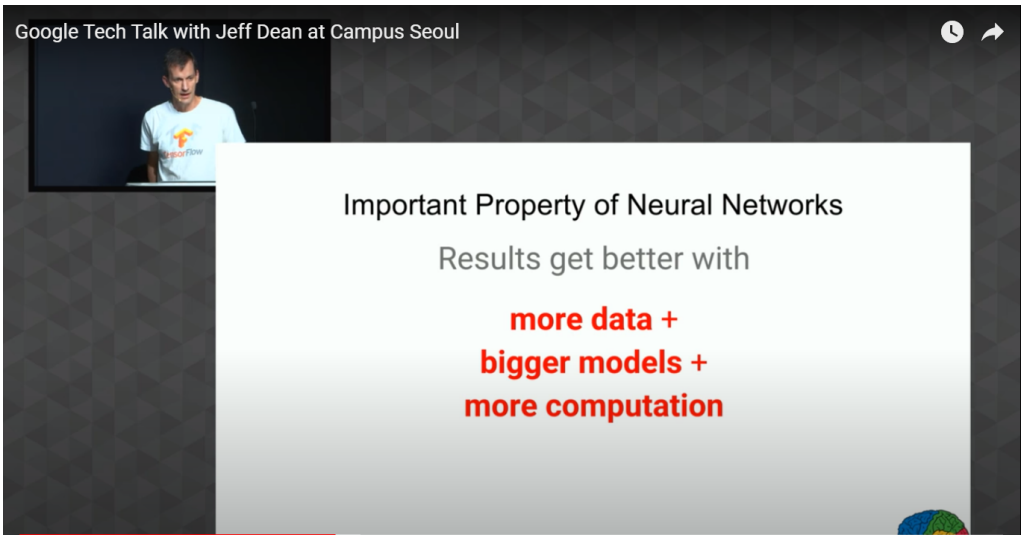
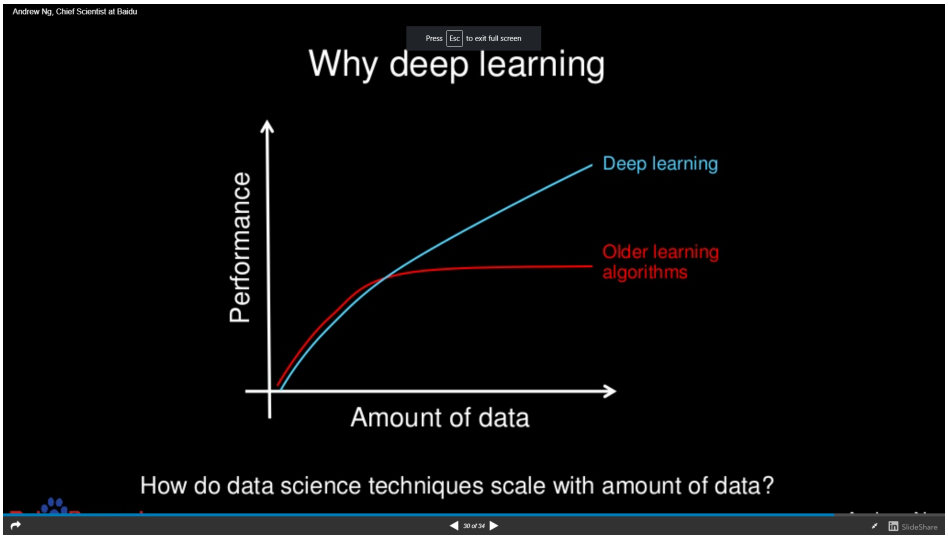


Figure 19: Performance vs. data scale for deep learning vs. older algorithms (Ng, 2015)



Tested neural network architectures:

Six different architectures were tested, combining LSTM, Bidirectional LSTM, Conv1D, Dense, Dropout, MaxPooling1D, and Flatten layers. The architectures ranged from simple (LSTM + Dropout + Dense) to complex (BiLSTM + Conv1D + Dense with multiple layers).

Figure 21: Neural network architectures tested

```
models = {
  "clf_LSTM_500n":          [LSTM(500), Dropout(0.5), Dense(nb_classes, "softmax")],
  "clf_BiLSTM_500n":        [Bidirectional(LSTM(500)), Dropout(0.5), Dense(...)],
  "clf_LSTM_500n_MultipleDense":    [LSTM(500), Dropout, Dense(30,"relu"), Dropout, Dense(...)],
  "clf_BiLSTM_500n_MultipleDense":  [BiLSTM(500), Dropout, Dense(30,"relu"), Dropout, Dense(...)],
  "clf_CNN1D_500n_BiLSTM_MultipleDense": [Conv1D(32), MaxPool, BiLSTM(500), Dense, Dropout, Dense],
  "clf_BiLSTM_500n_CNN1D_MultipleDense": [BiLSTM(500), Conv1D(32), MaxPool, Dense, Dropout, Flatten, Dense]
}
```

Key architecture choices: LSTM (Long Short-Term Memory) is a type of recurrent network that excels at recognizing sequential data. Bidirectional LSTM learns sequences both left-to-right and right-to-left. Dropout acts as regularization during training. Convolutional networks (Conv1D) apply filters across the feature space to capture patterns.

Data preparation for deep neural networks:

1. Target variable mapped same as MLR (1,2→0; 3→1; 4,5→2)
2. All letters converted to lowercase
3. Tokenization of 'reviewtext' using keras built-in method
4. Indexing of tokenized text (conversion to numeric values)
5. Padding all inputs to uniform dimension
6. Creating embedding matrix using pre-trained GloVe vectors (glove.42B.300d with 42 billion tokens, Stanford University)
7. Using categorical_crossentropy loss, softmax activation, categorical_accuracy metric
8. Callbacks: EarlyStopping (patience=5) and ReduceLROnPlateau (patience=2)

Model selection (5,000 data sample):

Figure 22: Results of deep neural network models using only 'reviewtext'

#	accuracy	precision	recall	f1-score	model
0	0.805	0.777	0.805	0.783	clf_BiLSTM_500n_CNN1D_MultipleDense
1	0.772	0.802	0.772	0.779	clf_CNN1D_500n_BiLSTM_MultipleDense
2	0.759	0.771	0.759	0.761	clf_BiLSTM_500n_MultipleDense
3	0.744	0.789	0.744	0.760	clf_LSTM_500n
4	0.711	0.812	0.711	0.750	clf_LSTM_500n_MultipleDense
5	0.713	0.794	0.713	0.746	clf_BiLSTM_500n

The two deepest models achieved the best results. The clf_BiLSTM_500n_CNN1D_MultipleDense model had the best F1 score (0.783) and was selected for further analysis.

When the same model was tested with both 'reviewtext' and 'category' variables, the results were poor (accuracy 0.137, F1 0.109). The model using only 'reviewtext' was therefore selected for final training on larger data.

Chapter V: Data-Driven Product

Subchapter 1: Hive Analysis

Rankings are presented at two levels of granularity: overall (top 5 positions) and per category (one product per category). Evaluation criteria, also for comparative purposes:

1. Most popular books (based on number of reviews)
2. Highest mean rating
3. Highest weighted average rating (as described in Chapter IV, Subchapter 1)

Table 1: Most popular books overall (top 5 most reviewed)

asin	category	count	title	mean
B00YN6XHMU	literature & fiction	13,442	Grey: Fifty Shades of Grey as told	4.29
B00DMCV7K0	literature & fiction	9,691	The Invention of Wings: A Novel	4.66
B00C2WDD5I	literature & fiction	8,437	The Atlantis Gene: A Thriller	4.09
B00XSSYR50	politics & social sci.	6,660	When Breath Becomes Air	4.74
B00571F26Y	health, fitness	5,341	Wheat Belly	4.44

Table 5: Highest-rated books by weighted average

asin	category	count	title	mean
B00EFWU9QY	children's ebooks	2,970	Rush Revere and the First Patriots	4.92
B00XSSYR50	politics & social sci.	6,660	When Breath Becomes Air	4.74
B00DMCV7K0	literature & fiction	9,691	The Invention of Wings	4.66
B00ESJ3S94	literature & fiction	2,702	Fueled (The Driven Series Book 2)	4.86
B001BPYMCU	business & money	4,752	The Total Money Makeover	4.69

The weighted measure results show a reasonable balance between popularity and rating. For example, in the 'literature & fiction' category, B00DMCV7K0 (The Invention of Wings) ranks highest with ~9.7k reviews and 4.66 mean rating, even though B00YN6XHMU (Grey) has more reviews (~13.5k) but a lower mean (4.28). This confirms the measure works as intended.

Subchapter 2: Comparison and Selection of Final Sentiment Analysis Model

Two models that achieved the best results were selected: Multinomial Logistic Regression in Spark, and deep neural networks trained using GPU acceleration on Google Colab.

Table 7: Differences in final models

Detail	BiLSTM + CNN1D	MLR
Data	~500K subset (Google Colab RAM limits)	All data (~3M)
Variables	'reviewtext' only	'reviewtext' and 'category'
Big Data tech	GPU-accelerated computation	Spark distributed processing

Table 8: Comparison of main evaluation metrics

Metric	BiLSTM + CNN1D	MLR
Accuracy	80.27%	**84.69%**
Precision	**86.01%**	81.45%
Recall	80.27%	**84.69%**
F1-score	**82.48%**	80.71%

Table 9: Confusion Matrix and Normalized Confusion Matrix

	BiLSTM + CNN1D			MLR		
	Pred 0	Pred 1	Pred 2	Pred 0	Pred 1	Pred 2
Class 0	4,526	1,447	520	14,125	2,593	33,628
Class 1	1,360	3,913	1,929	3,363	6,536	46,984
Class 2	1,817	7,722	51,766	2,692	3,281	491,108

Normalized Confusion Matrix

	BiLSTM + CNN1D			MLR		
	Pred 0	Pred 1	Pred 2	Pred 0	Pred 1	Pred 2
Class 0	69.71%	22.29%	8.01%	28.06%	5.15%	66.79%
Class 1	18.88%	54.33%	26.78%	5.91%	11.49%	82.60%
Class 2	2.96%	12.60%	84.44%	0.54%	0.66%	98.80%

The neural network model achieved a higher F1 score, although both models performed well. The MLR model shows higher recall, so depending on task requirements, it may be preferred. However, the normalized confusion matrix reveals that MLR assigns too much weight to the majority class rather than accurately reflecting the structure of the data – this is expected behavior with imbalanced datasets. The neural network model is much more balanced across classes, as class weights were applied. A necessary improvement would be applying similar class weights to the MLR model.

Figure 24: Final results for deep neural network model

	accuracy	precision	recall	f1-score	model
0	0.802733	0.860067	0.802733	0.824827	clf_BiLSTM_500n_CNN1D_MultipleDense

```
# confusion matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[ 4526  1447   520]
 [ 1360  3913  1929]
 [ 1817  7722 51766]]
```

Figure 25: Final results for MLR model in Spark

```
// Weighted stats
println(s"Weighted precision: ${metrics.weightedPrecision}")
println(s"Weighted recall: ${metrics.weightedRecall}")
println(s"Weighted F1 score: ${metrics.weightedFMeasure}")
println(s"Weighted false positive rate: ${metrics.weightedFalsePositiveRate}")
```

```
import org.apache.spark.mllib.evaluation.MulticlassMetrics
results: org.apache.spark.sql.DataFrame = [asin: string, reviewtext: string ... 4 more fields]
predictionAndLabels: org.apache.spark.rdd.RDD[(Double, Double)] = MapPartitionsRDD[39528] at rdd at <console>:199
metrics: org.apache.spark.mllib.evaluation.MulticlassMetrics = org.apache.spark.mllib.evaluation.MulticlassMetrics@ce39930
Confusion matrix:
14125.0  2593.0  33628.0
3363.0   6536.0  46984.0
2692.0   3281.0  491108.0
accuracy: Double = 0.846865019609141
Summary Statistics
Accuracy = 0.846865019609141
Weighted precision: 0.8144684381243038
Weighted recall: 0.8468650196091411
Weighted F1 score: 0.8070515918639329
Weighted false positive rate: 0.6202997707641787
```

Chapter VI: Final Conclusions

This project presented a multidimensional analysis of the Kindle dataset from the Amazon e-commerce website. In the first part, using Hive, a ranking of book titles was created based on a mixed measure taking into account both the rating and the number of reviews (popularity) of a given title. From an e-commerce business perspective, this is a very useful analysis, as it will allow users and potential customers to select titles that truly reflect both popularity and how well a given book appealed to those who have already read it. As a result, customers will be able to find interesting products faster, which in turn can contribute to increased sales, building positive customer relationships, and cultivating brand loyalty.

Subsequently, using machine learning methods in the context of Big Data, sentiment analysis was conducted and a general attempt was made to predict ratings based on review text and the category to which a given book belongs. Based on the results obtained, it was determined that the models are able to predict ratings well. The motivation for such analysis is that the description contains more information than the rating alone, and some customers may accidentally click an unintended rating. In a text review, the risk of such error is dramatically lower.

POTENTIAL NEXT STEPS

1. Building a streaming data acquisition process for real-time analysis, using technologies such as Spark Streaming or Kafka.

2. Automating parts of the current process that were performed manually, e.g., saving data to HDFS.

3. Using Bayesian estimation to create a more stable and systematic ranking tool.

4. Trying newer text vectorization methods such as ELMO and BERT.

5. Extending the project with relationship modeling in data using graph databases such as Neo4j, which could help draw new conclusions not visible in tabular analysis.

BIBLIOGRAPHY

Amazon AWS, n.d. *JSON SerDe Libraries.* Available at: https://docs.aws.amazon.com/athena/latest/ug/json-serde.html

Benjamin Bengfort, D.D.L., 2017. *Computing a Bayesian Estimate of Star Rating Means.* Medium.

Data-Flair Trainings Blog, n.d. *Hive Data Types – Primitive and Complex Data Types in Hive.*

Dean, J., 2016. *Large-Scale Deep Learning for Intelligent Computer Systems.* Seoul: Google.

Ganesan, K., 2019. *All you need to know about text preprocessing for NLP and Machine Learning.* Towards Data Science.

Geron, A., 2019. *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow.* 2nd ed. O'Reilly.

Ian Robinson, J.W.E.E., 2015. *Graph Databases – New Opportunities For Connected Data.* 2nd ed. O'Reilly Media.

Jeffrey Pennington, R.S.C.D.M., 2014. *GloVe: Global Vectors for Word Representation.* Stanford NLP.

Jianmo Ni, J.L.J.M., 2019. *Justifying recommendations using distantly-labeled reviews and fined-grained aspects.* EMNLP.

JSON Lines Org., n.d. *JSON Lines; Documentation for the JSON Lines text file format.*

Kane, F., 2017. *Taming Big Data with Apache Spark and Python.* Packt.

Kane, F., n.d. *The Ultimate Hands-On Hadoop: Tame your Big Data!* Udemy.

Marek Gagolewski, M.B.A.C., 2016. *Przetwarzanie i analiza danych w jezyku Python.* 1st ed. PWN.

Masurel, P., 2013. *Of bayesian average and star ratings.*

Ng, A., 2015. *What data scientists should know about deep learning.*

Szmit, R., n.d. *Apache Hadoop – presentation, Big Data postgraduate studies.*

The Apache Software Foundation, n.d. *https://hive.apache.org/*

APPENDICES

Appendix 1: Python source code generating smaller files

https://github.com/ambiernat/Big_Data/blob/main/File_Generator-OOP.ipynb

Appendix 2: Loading data from HDFS to Apache Hive

https://github.com/ambiernat/Big_Data/blob/main/read_data_from_HDFS_to_Hive.sql

Appendix 3: Reconnecting Kindle and meta_Kindle data in Hive

https://github.com/ambiernat/Big_Data/blob/main/read_data_from_HDFS_to_Hive.sql

Appendix 4: Data Exploration in Hive

https://github.com/ambiernat/Big_Data/blob/main/HIVE_4_exploratory_hive.sql

Appendix 5: Kindle data preprocessing

https://github.com/ambiernat/Big_Data/blob/main/HIVE_3_kindle_preproc.sql

Appendix 6: Meta Kindle data preprocessing

https://github.com/ambiernat/Big_Data/blob/main/HIVE_5_meta_kindle_preproc.sql

Appendix 7: Merging datasets for Hive, Spark Scala, Python

https://github.com/ambiernat/Big_Data/blob/main/HIVE_6_merge_datasets.sql

Appendix 8: Duplicates – ASIN and category

(see inline table in original)

Appendix 9: Final data cleaning for Hive analysis

https://github.com/ambiernat/Big_Data/blob/main/HIVE_7_preprocess4hive.sql

Appendix 10: Final Hive queries

https://github.com/ambiernat/Big_Data/blob/main/HIVE_8_hive_rankings.sql

Appendix 11: Hive – all results

(see original document)

Appendix 12: MLR and GBT models on data sample in Spark

https://github.com/ambiernat/Big_Data/blob/main/SparkML_Scala_BD_MLR_GBT_small.ipynb

Appendix 13: Deep NN model using 'reviewtext' for 5,000 data

https://github.com/ambiernat/Big_Data/blob/main/NLP_small_data_tidy1.ipynb

Appendix 14: NN model using 'reviewtext' and 'category' for 5,000 data

https://github.com/ambiernat/Big_Data/blob/main/NLP_small_data_reviewtext_category.ipynb

Appendix 15: Deep NN model using 'reviewtext' for 500,000 data

https://github.com/ambiernat/Big_Data/blob/main/NLP_big_data_500k_final_GloVe.ipynb

Appendix 16: MLR model in Spark trained on all data

https://github.com/ambiernat/Big_Data/blob/main/SparkML_Scala_BD_MLR_allData.ipynb